

Data Management Technology

Unit 8: Individual Consolidation Activities

Activity 1. Concept Review Questions

Differences between relational, document-based and graph-based databases.

Relational databases store data in tables containing rows and columns of linked data with fixed schemas. Tables typically include primary keys, which are unique for each entry, and foreign keys which link related tables. Examples of relational database types are MySQL and PostgreSQL.

Document-based databases store data in document modes similar to JSON documents, with entries containing keys and values. The schemas are flexible, meaning that all keys need not exist for all entries. An example is MongoDB.

Graph-based databases store data using nodes (entities) and edges (relationships). This works well for databases holding interconnected and versatile data, where certain relationships exist for some entities but not for others.

Nodes represent entities in the database (e.g. users, workplaces, schools) and can hold properties (e.g. gender, birthday, hobbies).

Edges show links between the nodes (e.g. 'A is friends with X', 'A attends Y school' and 'A lives in Z city'). An example is Neo4j.

MongoDB is better than a relational database where:

- **Horizontal Scaling is required:** A document-based database works well when data needs to be scaled horizontally, for example by adding characteristics to existing entities. (Relational structures, on the other hand are better at scaling vertically)
- **Flexible Schemas are involved:** MongoDB works well for flexible schemas, wherein some keys do not exist for all entities. An example is a library database where all book entities have a first edition publication date, but not all entities have a second edition publication date.
- **High Write Throughput is required:** MongoDB works well for handling high volumes of insert and update operations quickly. This is relevant for handling data collected by IoT (internet of things) devices, where sensors collect large amounts of data at a quick pace.

Components of a Neo4j graph

- **Node:** An entity. (E.g. a person registered in a school database).
- **Label:** This is a tag for a node that describes it. A node can have multiple labels (e.g., a school database could classify each person as a teacher or student, and each student as a junior or senior).
- **Relationship:** This details a link between nodes (e.g. students enrolled in courses).
- **Properties:** Data in key-value format that can be attached to a node or relationship (e.g. {age: 16} on a student's node).
- **Transversal:** The action of following relationships. For example, on a social media site, following a linked line of "friends" (A is friends with B, B is friends with C, C is friends with D). This will be useful for recommending users that A probably knows.

Activity 2. MongoDB Basics

Install MongoDB locally or access a cloud-based MongoDB service.

Create a MongoDB collection for a hypothetical application (e.g., a movie database).

Populate it with JSON-like documents, including nested and array data structures.

Write MongoDB queries to:

Retrieve specific documents using filters (e.g., find movies by genre or release year).

Update nested fields in documents.

Use aggregation pipelines to group and analyze data (e.g., find the average rating of movies by genre).

Mongo DB queries

a). Retrieve specific documents using filters (e.g., find movies by genre or release year).

Finding a book by id (using the query bar)

```
{id: "BK005"}
```

Find books released in the 1950s

```
({year: {$gte: 1950, $lte: 1959}})
```

b). Update nested fields in documents.

Updating Quantity of all books to 125 units:

```
{$set: {quantity: 125},}
```

c). Use aggregation pipelines to group and analyze data (e.g., find the average rating of movies by genre).

Stage 1: \$match

```
{year: {$gte: 1950, $lte: 1959}}
```

Stage 2: \$group

```
{_id: null, avgQuantity: {$avg: "$quantity"}}
```

(Finds average quantity for all values filtered in stage 1)

Activity 3. Introduction to Neo4j

(Task completed on [Neo4j Sandbox](#))

Install Neo4j Desktop or use the Neo4j Sandbox.

Create a small graph representing a social network, with nodes for "Person" and "Interest" and relationships like "FRIEND" or "LIKES."

Creating User entries

```
CREATE (u:User {user_id: 5, user_name: "JulliettaJohnson", user_age: 21, interest: "Drama"})
```

Creating Interest entries:

```
CREATE
```

```
(i1: Interest {interest_id: 2, interest_name : "Programming"}),
```

```
(i2: Interest { interest_id: 3, interest_name : "Hiking"})
```

Use Cypher queries to:

i. Retrieve all friends of a specific person.

Retrieving friends of the user Peter1998 (whose user id is 3):

```
MATCH (u:User {user_id: 3})-[:FRIENDS_WITH]->(friend:User)
```

```
RETURN friend
```

ii. Find people who like the same interests.

Finding people who like chess

```
MATCH (u:User)-[:LIKES]->(i:Interest {interest_id: 1})
RETURN u
```

iii. Identify the shortest path between two people in the network.

```
MATCH p = shortestPath(
(u1:User {user_id: 1})-[:FRIENDS_WITH*]-(u2:User {user_id: 3})
)
RETURN p
```

(This applies Neo4j's built-in function for the shortest path. The result shows that the shortest path is through a single mutual friend)